# Telekommunikációs Hálózatok

*Számítógépes Hálózatok*

2. gyakorlat

# Ismétlés:
# File beolvasás

```
f = open("demofile.txt", "r")
print(f.read())   #read(x) -> x karakter beolvasása (ha
                             szövegként van megnyitva)
for x in f:
  print(x)
f.close()
f.readline() #sor
```

> "When size is omitted or negative, the entire contents of the file will be read and returned; it's your problem if the file is twice as large as your machine's memory."
> – python.org

```
with open('alma.txt', 'r') as f:
   for line in f:
      print( line.strip().split(',') )
```

```
f = open("demofile.txt", "w")        # w-write, a-append

f.write(„Bla Bla")
```

olvasása, kezelése, írása

# BINÁRIS FÁJLOK

# Fájl átvitel

- fájl bináris megnyitása

```
with open („input.txt”, „rb”) as f:
                …
```

- read(x)
  - x byte beolvasása (ha binárisra van megnyitva)

```
                …
                f.read(128)    #128 byte-ot fog beolvasni
```

„When size is omitted or negative, the entire contents of the file will be read and returned; it’s your problem if the file is twice as large as your machine’s memory. „ - python.org

# Fájlkezelés

- SEEK: a fájlobjektumnak az aktuális pozíciója:

```
with open('alma.txt', 'r') as f:
  sor = f.readline()
  print('jelenlegi sor', sor)     # jelenlegi sor 1. sor

  sor = f.readline()
  print('jelenlegi sor', sor)     # jelenlegi sor 2. sor

  f.seek(0, 0)                    # f.seek(offset, whence)

  sor = f.readline()
  print('jelenlegi sor', sor)     # jelenlegi sor 1. sor
```

- offset: olvasás/írás mutató pozíciója a fájlban
- (whence: alapért. 0: abszolút poz., 1: relatív aktuális poz.-hoz, 2: rel. a fájl végéhez)

stringek, byte-stringek

# STRUKTÚRA

# Bytesorozat vs string

- String → Byte sorozat

```
str = 'hello'
str.encode()            # b’hello’
    #VAGY
str = b'hello'          # b’hello’
```

- Byte sorozat → String

```
str.decode()                    # ’hello’
```

# Struktúra

- Binárissá alakítjuk az adatot

```
import struct
values = (1, 'ab'.encode(), 2.7)
packer = struct.Struct(,i 2s f')                #Int, char[2], float
packed_data = packer.pack(*values)
```

- Visszalakítjuk a kapott üzenetet

```
import struct
unpacker = struct.Struct(,i 2s f')              #struct.calcsize(unpacker)
unpacked_data = unpacker.unpack(data)
```

- megj.: integer 1 – 4 byte, stringként 1 byte, azaz hatékonyabb stringként átküldeni.

# Struktúra jellemzői

- Mire kell figyelni?
  - A Struct formátumnál az „Xs” (pl. „3s”) X db. bájtból álló **bájtliterált** jelent (pl. **b'abc'**)

```
import struct
values = (1, 'ab', 2.7)
packer = struct.Struct(,i 2s f')
packed_data = packer.pack(*values)
# HIBA: struct.error: argument for 's' must be a bytes object
# JÓ megoldás:
values = (1, b'ab', 2.7) # vagy values = (1, 'ab'.encode(), 2.7)
…
```

# Struktúra explicit hosszal

- String → Byte sorozat

```
import struct
str = „hello”
str.encode()          # b’hello’

Struct.pack(’8s’,str)        #b’hello\x00\x00\x00’
```

- Byte sorozat → String

```
import struct
d = Struct.pack(’8s’,str)    #b’hello\x00\x00\x00’

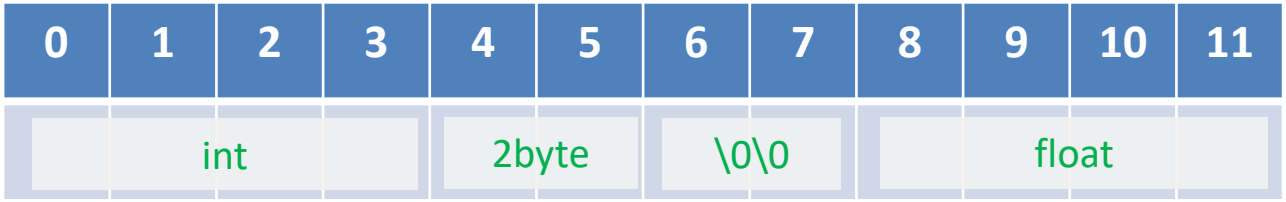
d.decode().strip(’\x00’)     #’hello’
```

# Struktúra jellemzői

- Mire kell figyelni?
  - A struktúra mérete byte-ban:

```
import struct
packer = struct.Struct(,i 2s f')
print(struct.calcsize(,i 2s f '))
print(packer.size)
# 12
```

    - i: int size = 4, 2s: 2 bytes, f: float size = 4, 4+2+4 ≠ **12** ???
  - Az int/float-ot úgy igazítja, hogy a kezdő pozíciója 4-gyel osztható legyen



# Struktúra: típusok és méretek

| Character | Byte order |
|---|---|
| @ | native |
| = | native |
| < | little-endian |
| > | big-endian |
| ! | network (= big-endian) |

| Format | C Type | Python type | Standard size |
|---|---|---|---|
| x | pad byte | no value | |
| c | char | bytes of length 1 | 1 |
| b | signed char | integer | 1 |
| B | unsigned char | integer | 1 |
| ? | _Bool | bool | 1 |
| h | short | integer | 2 |
| H | unsigned short | integer | 2 |
| i | int | integer | 4 |
| I | unsigned int | integer | 4 |
| l | long | integer | 4 |
| L | unsigned long | integer | 4 |
| q | long long | integer | 8 |
| Q | unsigned long long | integer | 8 |
| n | ssize_t | integer | |
| N | size_t | integer | |
| e | (6) | float | 2 |
| f | float | float | 4 |
| d | double | float | 8 |
| s | char[] | bytes | |
| p | char[] | bytes | |
| P | void* | integer | |

# Fájlkezelés

- Bináris fájl és a SEEK

```
import struct

packer = struct.Struct('i 3s i')

with open('dates.bin', 'wb') as f:
  for i in range(5):
    values = (2020+i, b'jan', 10+i)
    packed_data = packer.pack(*values)
    f.write(packed_data)

with open('dates.bin', 'rb') as f:   # olvassuk be
  f.seek(packer.size*4)              az utolsó sort
  data = f.read(packer.size)
  print(packer.unpack(data))

### output: (2024, b'jan’, 14)
```

ping

# HÁLÓZATI ESZKÖZÖK I.

# Ping a hoszt elérhetőségének ellenőrzésére és a Round Trip Time (RTT) méréséhez

## Linuxon

```
lakis@dpdk-pktgen:~$ ping -c 3 berkeley.edu
PING berkeley.edu (35.163.72.93) 56(84) bytes of data.
64 bytes from ec2-35-163-72-93.us-west-2.compute.amazonaws.com (35.163.72.93): icmp_seq=1 ttl=23 time=194 ms
64 bytes from ec2-35-163-72-93.us-west-2.compute.amazonaws.com (35.163.72.93): icmp_seq=2 ttl=23 time=194 ms
64 bytes from ec2-35-163-72-93.us-west-2.compute.amazonaws.com (35.163.72.93): icmp_seq=3 ttl=23 time=193 ms

--- berkeley.edu ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 193.093/193.937/194.428/0.786 ms
```

# Ping a hoszt elérhetőségének ellenőrzésére és a Round Trip Time (RTT) méréséhez

**Windowson**

```
C:\Users\laki>ping -n 3 berkeley.edu

Pinging berkeley.edu [35.163.72.93] with 32 bytes of data:
Reply from 35.163.72.93: bytes=32 time=200ms TTL=39
Reply from 35.163.72.93: bytes=32 time=201ms TTL=39
Reply from 35.163.72.93: bytes=32 time=200ms TTL=39

Ping statistics for 35.163.72.93:
    Packets: Sent = 3, Received = 3, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 200ms, Maximum = 201ms, Average = 200ms
```

traceroute

# HÁLÓZATI ESZKÖZÖK II.

# traceroute (linux) – tracert (windows)

Cél a hálózati útvonal meghatározása egy célállomás felé!

```
lakis@dpdk-pktgen:~$ traceroute berkeley.edu
traceroute to berkeley.edu (35.163.72.93), 30 hops max, 60 byte packets
 1  192.168.0.192 (192.168.0.192)  0.292 ms  0.344 ms  0.390 ms
 2  ikoktatok-gate.inf.elte.hu (157.181.167.254)  1.251 ms  1.250 ms  1.265 ms
 3  taurus.centaur-taurus.elte.hu (157.181.126.134)  5.180 ms  5.267 ms  5.325 ms
 4  fw1.firewall.elte.hu (157.181.141.145)  1.271 ms  1.358 ms  1.299 ms
 5  taurus.fw1.fw.backbone.elte.hu (192.153.18.146)  5.626 ms  5.356 ms  5.395 ms
 6  rtr.hbone-elte.elte.hu (157.181.141.9)  2.229 ms  1.245 ms  1.749 ms
 7  tg0-0-0-14.rtr2.vh.hbone.hu (195.111.100.47)  2.377 ms  2.415 ms  2.407 ms
 8  be1.rtr1.vh.hbone.hu (195.111.96.56)  1.945 ms  1.642 ms  1.877 ms
 9  bpt-b4-link.telia.net (80.239.195.56)  1.626 ms  1.581 ms  1.097 ms
10  win-bb2-link.telia.net (62.115.143.116)  196.574 ms win-bb2-link.telia.net (213.155.137.38)  196.993 ms
win-bb2-link.telia.net (213.155.135.222)  180.071 ms
11  ffm-bb4-link.telia.net (62.115.133.79)  199.425 ms  199.232 ms *
12  * * *
13  prs-bb3-link.telia.net (62.115.137.114)  180.494 ms  179.986 ms *
14  sjo-b21-link.telia.net (62.115.119.229)  197.252 ms  197.249 ms  197.264 ms
15  * a100row-ic-300117-sjo-b21.c.telia.net (213.248.87.118)  196.555 ms *
16  nyk-bb4-link.telia.net (62.115.142.222)  180.081 ms 54.240.242.148 (54.240.242.148)  200.986 ms
54.240.242.88 (54.240.242.88)  201.877 ms
17  54.240.242.161 (54.240.242.161)  200.935 ms * *
18  * * *
19  * * *
```

**Linuxon**

# traceroute (linux) – tracert (windows)

Cél a hálózati útvonal meghatározása egy célállomás felé!

**Windowson**

```
C:\Users\laki>tracert berkeley.edu

Tracing route to berkeley.edu [35.163.72.93]
over a maximum of 30 hops:

  1     1 ms    <1 ms    <1 ms  dlinkrouter [192.168.0.1]
  2    24 ms     6 ms    60 ms  10.0.0.85
  3    54 ms    18 ms    13 ms  fibhost-66-110-33.fibernet.hu [85.66.110.33]
  4    13 ms    14 ms    13 ms  ae0.info-c1.invitech.hu [213.163.54.245]
  5    13 ms    12 ms    17 ms  te0-0-2-3.nr11.b020698-1.bud01.atlas.cogentco.com [149.6.182.13]
  6    13 ms    13 ms    16 ms  te0-0-2-1.agr11.bud01.atlas.cogentco.com [154.25.3.237]
  7    15 ms    13 ms    12 ms  be3272.ccr31.bud01.atlas.cogentco.com [154.54.59.197]
  8    17 ms    16 ms    19 ms  be3263.ccr22.bts01.atlas.cogentco.com [154.54.59.177]
  9    22 ms    22 ms    21 ms  be3045.ccr21.prg01.atlas.cogentco.com [154.54.59.105]
 10    29 ms    30 ms    27 ms  be3027.ccr41.ham01.atlas.cogentco.com [130.117.1.205]
 11    41 ms    36 ms    41 ms  be2815.ccr41.ams03.atlas.cogentco.com [154.54.38.205]
 12   134 ms   136 ms   133 ms  be12194.ccr41.lon13.atlas.cogentco.com [154.54.56.93]
 13   133 ms   136 ms   132 ms  be2982.ccr31.bos01.atlas.cogentco.com [154.54.1.117]
 14   135 ms   134 ms   137 ms  be3599.ccr21.alb02.atlas.cogentco.com [66.28.4.237]
 15   134 ms   134 ms   135 ms  be2878.ccr21.cle04.atlas.cogentco.com [154.54.26.129]
 16   136 ms   136 ms   134 ms  be2717.ccr41.ord01.atlas.cogentco.com [154.54.6.221]
 17   148 ms   147 ms   146 ms  be2831.ccr21.mci01.atlas.cogentco.com [154.54.42.165]
 18   158 ms   159 ms   159 ms  be3035.ccr21.den01.atlas.cogentco.com [154.54.5.89]
 19   168 ms   169 ms   167 ms  be3037.ccr21.slc01.atlas.cogentco.com [154.54.41.145]
 20   183 ms   183 ms   183 ms  be3109.ccr21.sfo01.atlas.cogentco.com [154.54.44.137]
 21   186 ms   187 ms   184 ms  be3669.ccr41.sjc03.atlas.cogentco.com [154.54.43.10]
 22   184 ms   186 ms   185 ms  38.88.224.218
 23     *        *        *     Request timed out.
 24     *        *        *     Request timed out.
 25     *        *        *     Request timed out.
```

Tcpdump, Wireshark

# HÁLÓZATI FORGALOM

# NC-NetCat (SoCat), avagy hálózati svájcibicska

# szerver imitálása

```
nc -l -p 1234
```

# kliens imitálása

```
nc destination_host 1234
```

NetCat TUTORIAL:
https://www.binarytides.com/netcat-tutorial-for-beginners

SoCat TUTORIAL:
https://blog.rootshell.be/2010/10/31/socat-another-network-swiss-armyknife

# Tcpdump – hálózati forgalomfigyelés

```
lakis@dpdk-switch:~$ sudo tcpdump -i enp8s0
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on enp8s0, link-type EN10MB (Ethernet), capture size 262144 bytes
09:15:26.376139 IP 192.168.0.102.ssh > oktnb35.inf.elte.hu.55015: Flags [P.], seq 4154664816:4154665024, ack 289117644, win
09:15:26.376403 IP 192.168.0.102.43549 > 192.168.0.192.domain: 52681+ PTR? 35.167.181.157.in-addr.arpa. (45)
09:15:26.376994 IP 192.168.0.192.domain > 192.168.0.102.43549: 52681* 1/0/0 PTR oktnb35.inf.elte.hu. (78)
09:15:26.377100 IP 192.168.0.102.57511 > 192.168.0.192.domain: 64457+ PTR? 102.0.168.192.in-addr.arpa. (44)
09:15:26.377645 IP 192.168.0.192.domain > 192.168.0.102.57511: 64457 NXDomain 0/1/0 (79)
09:15:26.377723 IP 192.168.0.102.49012 > 192.168.0.192.domain: 6981+ PTR? 192.0.168.192.in-addr.arpa. (44)
09:15:26.377851 IP 192.168.0.102.ssh > oktnb35.inf.elte.hu.55015: Flags [P.], seq 208:400, ack 1, win 384, length 192
09:15:26.378180 IP 192.168.0.192.domain > 192.168.0.102.49012: 6981 NXDomain 0/1/0 (79)
09:15:26.378267 IP 192.168.0.102.ssh > oktnb35.inf.elte.hu.55015: Flags [P.], seq 400:976, ack 1, win 384, length 576
09:15:26.378291 IP 192.168.0.102.ssh > oktnb35.inf.elte.hu.55015: Flags [P.], seq 976:1248, ack 1, win 384, length 272
09:15:26.378340 IP 192.168.0.102.ssh > oktnb35.inf.elte.hu.55015: Flags [P.], seq 1248:1600, ack 1, win 384, length 352
09:15:26.378387 IP 192.168.0.102.ssh > oktnb35.inf.elte.hu.55015: Flags [P.], seq 1600:1776, ack 1, win 384, length 176
09:15:26.378440 IP 192.168.0.102.ssh > oktnb35.inf.elte.hu.55015: Flags [P.], seq 1776:1952, ack 1, win 384, length 176
09:15:26.378489 IP 192.168.0.102.ssh > oktnb35.inf.elte.hu.55015: Flags [P.], seq 1952:2128, ack 1, win 384, length 176
09:15:26.378538 IP 192.168.0.102.ssh > oktnb35.inf.elte.hu.55015: Flags [P.], seq 2128:2304, ack 1, win 384, length 176
09:15:26.378587 IP 192.168.0.102.ssh > oktnb35.inf.elte.hu.55015: Flags [P.], seq 2304:2480, ack 1, win 384, length 176
09:15:26.378636 IP 192.168.0.102.ssh > oktnb35.inf.elte.hu.55015: Flags [P.], seq 2480:2656, ack 1, win 384, length 176
09:15:26.378685 IP 192.168.0.102.ssh > oktnb35.inf.elte.hu.55015: Flags [P.], seq 2656:2832, ack 1, win 384, length 176
09:15:26.378734 IP 192.168.0.102.ssh > oktnb35.inf.elte.hu.55015: Flags [P.], seq 2832:3008, ack 1, win 384, length 176
09:15:26.378783 IP 192.168.0.102.ssh > oktnb35.inf.elte.hu.55015: Flags [P.], seq 3008:3184, ack 1, win 384, length 176
09:15:26.378832 IP 192.168.0.102.ssh > oktnb35.inf.elte.hu.55015: Flags [P.], seq 3184:3360, ack 1, win 384, length 176
```

# Tcpdump – szűrések (pl. protokollra)

```
lakis@dpdk-switch:~$ sudo tcpdump -i enp8s0 icmp
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on enp8s0, link-type EN10MB (Ethernet), capture size 262144 bytes
09:16:49.470737 IP dpdk-pktgen > 192.168.0.102: ICMP echo request, id 5668, seq 1, length 64
09:16:49.470766 IP 192.168.0.102 > dpdk-pktgen: ICMP echo reply, id 5668, seq 1, length 64
09:16:50.471818 IP dpdk-pktgen > 192.168.0.102: ICMP echo request, id 5668, seq 2, length 64
09:16:50.471834 IP 192.168.0.102 > dpdk-pktgen: ICMP echo reply, id 5668, seq 2, length 64
09:16:51.471716 IP dpdk-pktgen > 192.168.0.102: ICMP echo request, id 5668, seq 3, length 64
09:16:51.471732 IP 192.168.0.102 > dpdk-pktgen: ICMP echo reply, id 5668, seq 3, length 64
09:16:52.471713 IP dpdk-pktgen > 192.168.0.102: ICMP echo request, id 5668, seq 4, length 64
09:16:52.471729 IP 192.168.0.102 > dpdk-pktgen: ICMP echo reply, id 5668, seq 4, length 64
09:16:53.471720 IP dpdk-pktgen > 192.168.0.102: ICMP echo request, id 5668, seq 5, length 64
09:16:53.471736 IP 192.168.0.102 > dpdk-pktgen: ICMP echo reply, id 5668, seq 5, length 64
```

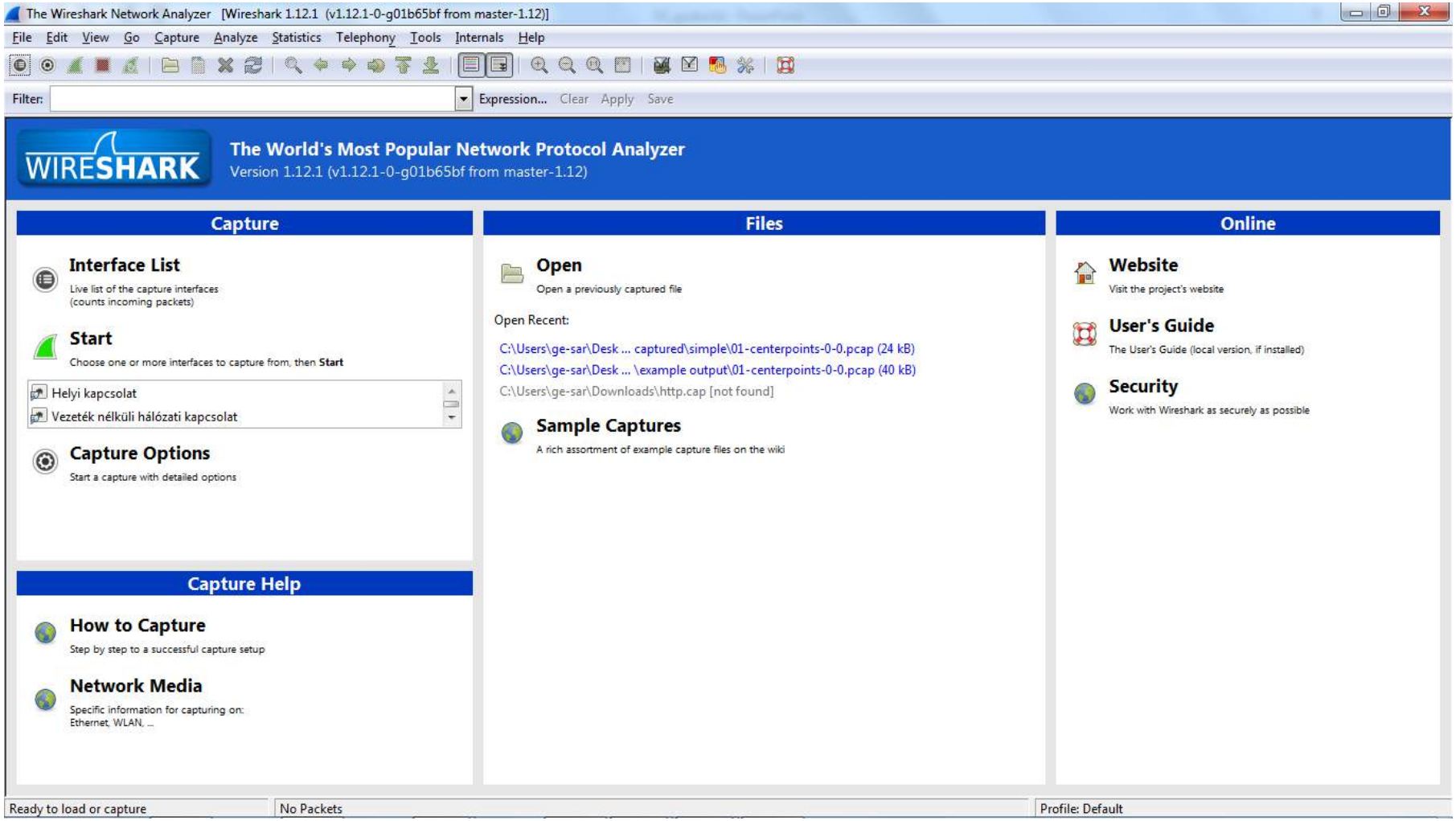
# Tcpdump – host és portszűrés

```
lakis@dpdk-switch:~$ sudo tcpdump -i enp8s0 host 192.168.0.101 and port 1111
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on enp8s0, link-type EN10MB (Ethernet), capture size 262144 bytes
09:20:23.289035 IP dpdk-pktgen.48524 > 192.168.0.102.1111: Flags [S],
seq 1544265047, win 29200, options [mss 1460,sackOK,TS val 409718781
ecr 0,nop,wscale 7],
length 0
09:20:23.289067 IP 192.168.0.102.1111 > dpdk-pktgen.48524: Flags [R.],
seq 0, ack 1544265048, win 0, length 0
```

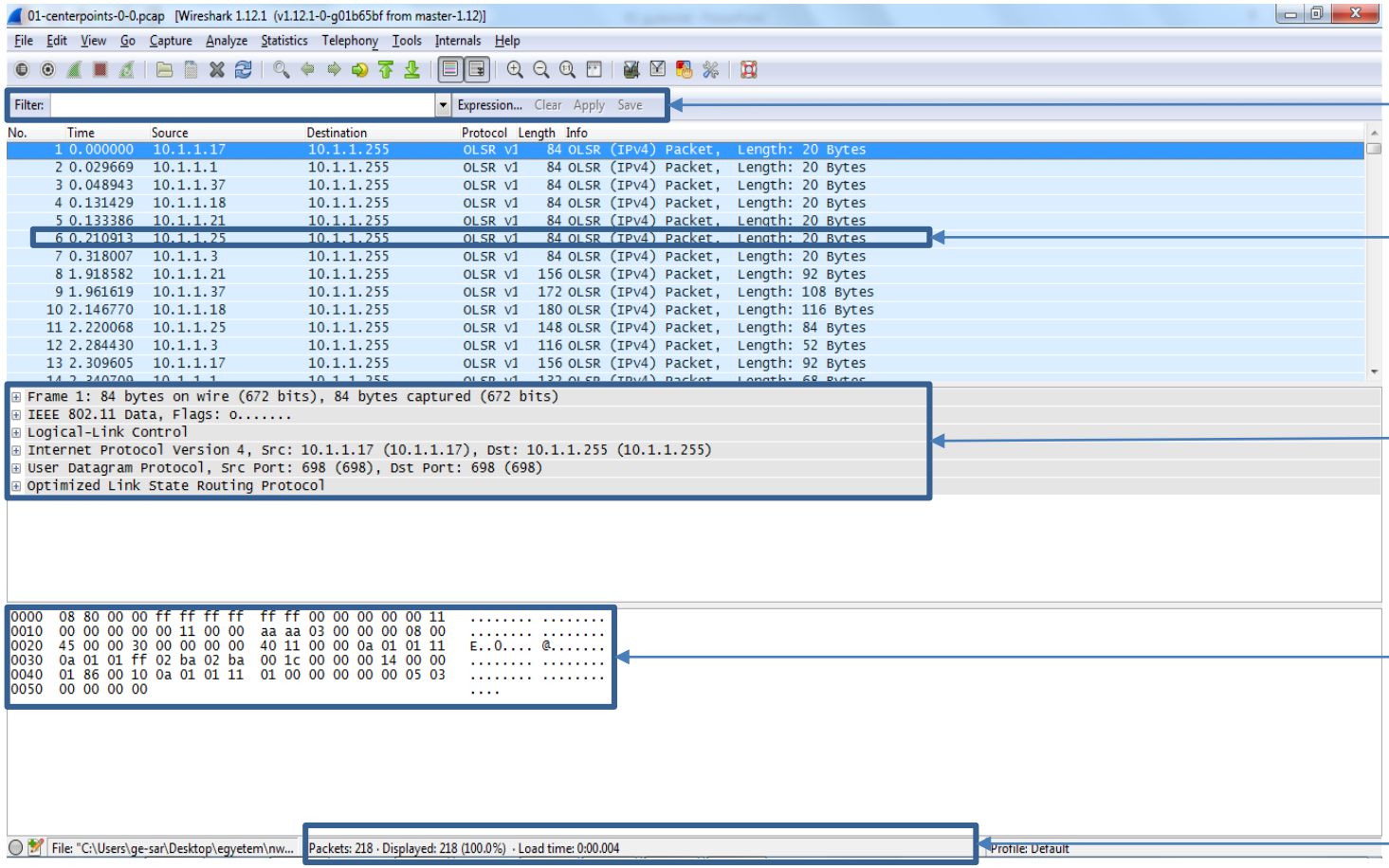
# Tcpdump – pcap fájlba mentés

```
lakis@dpdk-switch:~$ sudo tcpdump -w test.pcap -i enp8s0
tcpdump: listening on enp8s0, link-type EN10MB (Ethernet),
capture size 262144 bytes
^C4 packets captured
6 packets received by filter
0 packets dropped by kernel
lakis@dpdk-switch:~$ tcpdump -r test.pcap
reading from file test.pcap, link-type EN10MB (Ethernet)
09:31:32.000164 IP 192.168.0.102.ssh > oktnb35.inf.elte.hu.55015:
Flags [P.], seq 4154857792:4154857936, ack 289145644, win 384, length 144
09:31:32.060031 IP oktnb35.inf.elte.hu.55015 > 192.168.0.102.ssh:
Flags [.], ack 144, win 3542, length 0
09:31:34.354029 IP 192.168.0.192.48309 > 255.255.255.255.7437: UDP, length 173
09:31:37.377992 IP 192.168.0.192.48309 > 255.255.255.255.7437: UDP, length 173
```

# Wireshark


The Wireshark Network Analyzer [Wireshark 1.12.1 (v1.12.1-0-g01b65bf from master-1.12)]
File Edit View Go Capture Analyze Statistics Telephony Tools Internals Help
Filter: Expression... Clear Apply Save
WIRESHARK
The World's Most Popular Network Protocol Analyzer
Version 1.12.1 (v1.12.1-0-g01b65bf from master-1.12)
Capture
Interface List
Live list of the capture interfaces
(counts incoming packets)
Start
Choose one or more interfaces to capture from, then Start
Helyi kapcsolat
Vezeték nélküli hálózati kapcsolat
Capture Options
Start a capture with detailed options
Capture Help
How to Capture
Step by step to a successful capture setup
Network Media
Specific information for capturing on:
Ethernet, WLAN, ...
Files
Open
Open a previously captured file
Open Recent:
C:\Users\ge-sar\Desk ... captured\simple\01-centerpoints-0-0.pcap (24 kB)
C:\Users\ge-sar\Desk ... \example output\01-centerpoints-0-0.pcap (40 kB)
C:\Users\ge-sar\Downloads\http.cap [not found]
Sample Captures
A rich assortment of example capture files on the wiki
Online
Website
Visit the project's website
User's Guide
The User's Guide (local version, if installed)
Security
Work with Wireshark as securely as possible
Ready to load or capture
No Packets
Profile: Default

# Wireshark




Szűrők definiálására alkalmas input eszközök

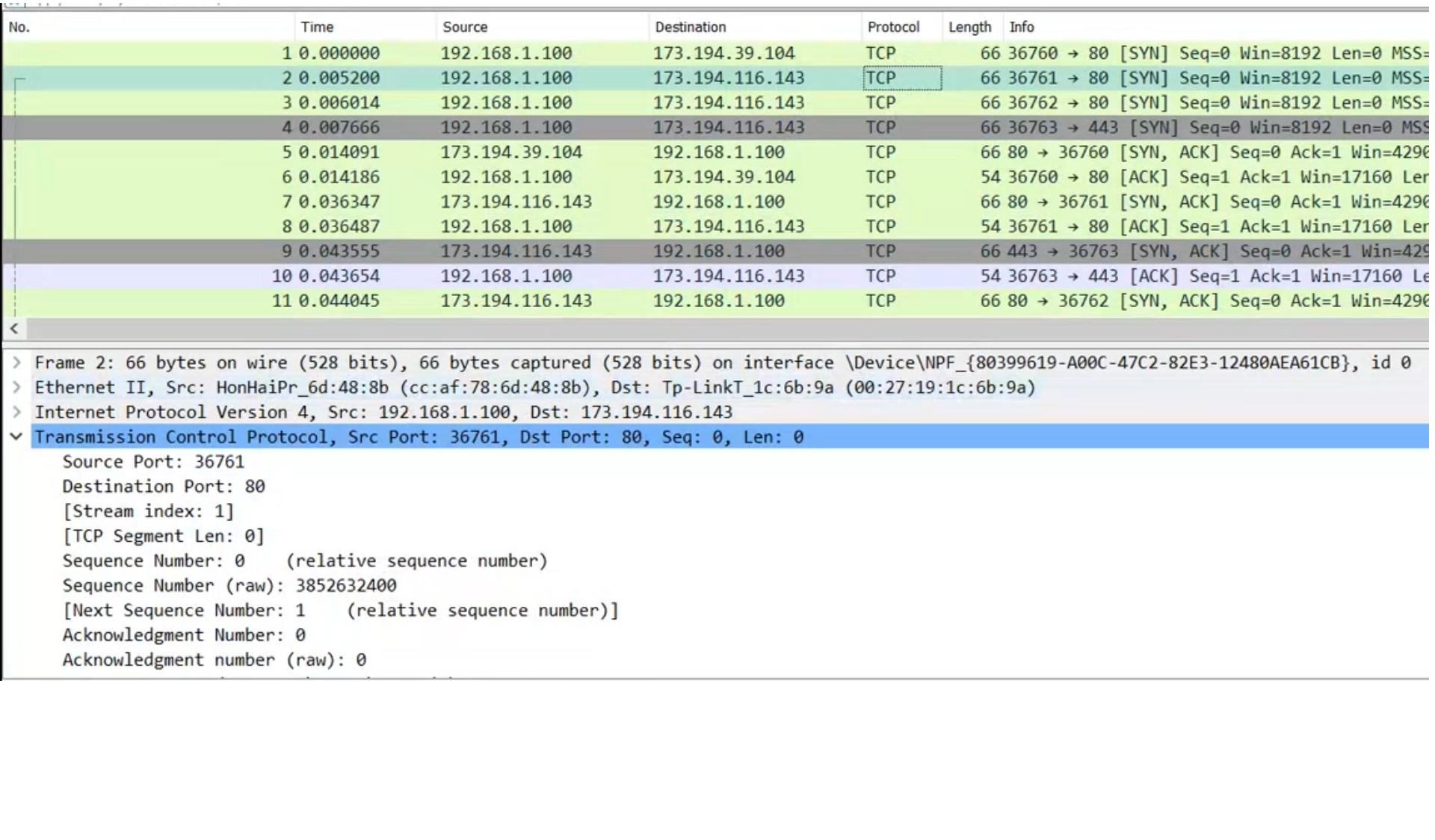
Csomag összefoglaló nézete

Kiválasztott csomag hierarchikus nézet

Kiválasztott csomag bájt-alapú nézet

Szűrés statisztikái

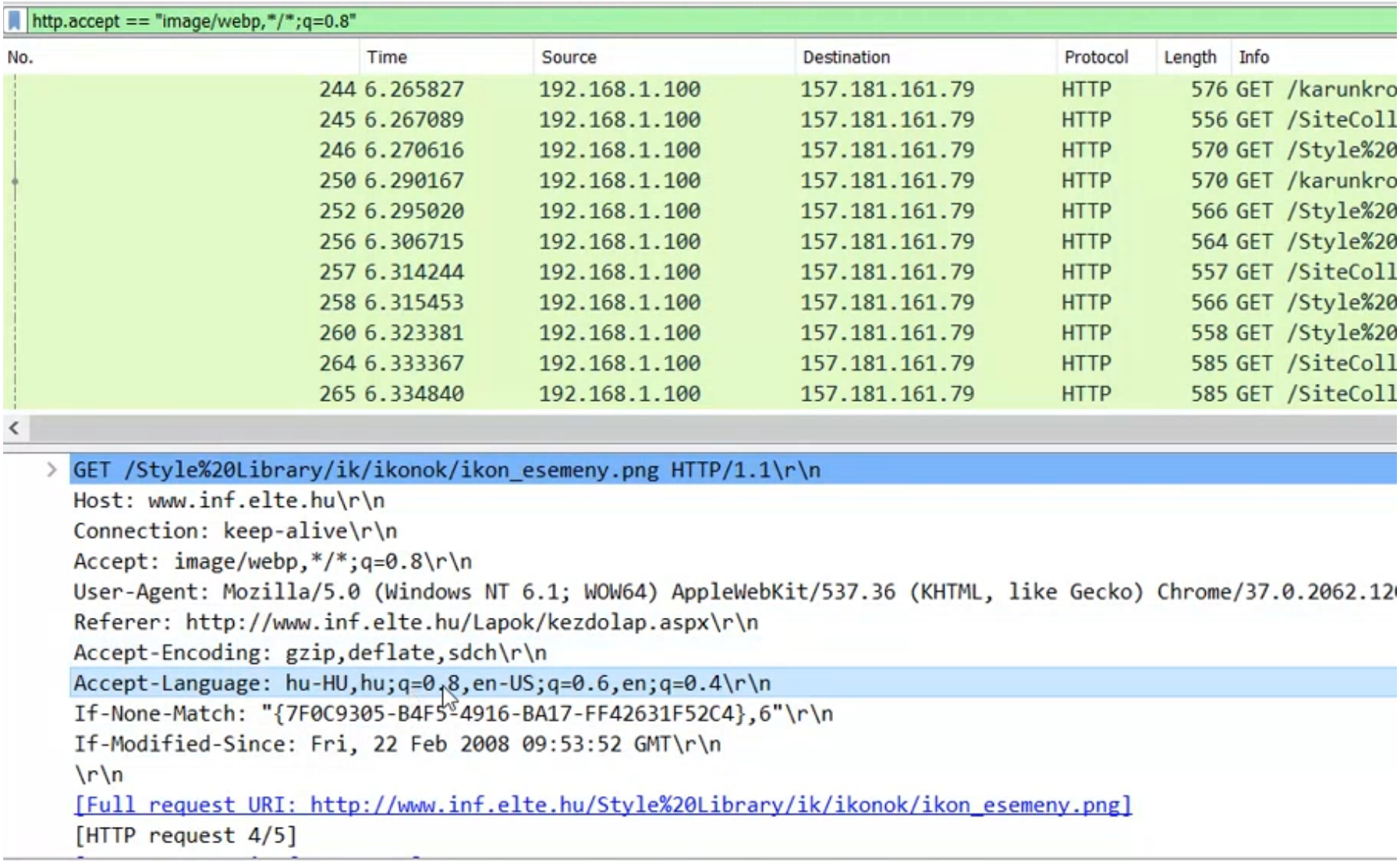
# Wireshark



| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 1 | 0.000000 | 192.168.1.100 | 173.194.39.104 | TCP | 66 | 36760 → 80 [SYN] Seq=0 Win=8192 Len=0 MSS= |
| 2 | 0.005200 | 192.168.1.100 | 173.194.116.143 | TCP | 66 | 36761 → 80 [SYN] Seq=0 Win=8192 Len=0 MSS= |
| 3 | 0.006014 | 192.168.1.100 | 173.194.116.143 | TCP | 66 | 36762 → 80 [SYN] Seq=0 Win=8192 Len=0 MSS= |
| 4 | 0.007666 | 192.168.1.100 | 173.194.116.143 | TCP | 66 | 36763 → 443 [SYN] Seq=0 Win=8192 Len=0 MSS |
| 5 | 0.014091 | 173.194.39.104 | 192.168.1.100 | TCP | 66 | 80 → 36760 [SYN, ACK] Seq=0 Ack=1 Win=4290 |
| 6 | 0.014186 | 192.168.1.100 | 173.194.39.104 | TCP | 54 | 36760 → 80 [ACK] Seq=1 Ack=1 Win=17160 Len |
| 7 | 0.036347 | 173.194.116.143 | 192.168.1.100 | TCP | 66 | 80 → 36761 [SYN, ACK] Seq=0 Ack=1 Win=4290 |
| 8 | 0.036487 | 192.168.1.100 | 173.194.116.143 | TCP | 54 | 36761 → 80 [ACK] Seq=1 Ack=1 Win=17160 Len |
| 9 | 0.043555 | 173.194.116.143 | 192.168.1.100 | TCP | 66 | 443 → 36763 [SYN, ACK] Seq=0 Ack=1 Win=429 |
| 10 | 0.043654 | 192.168.1.100 | 173.194.116.143 | TCP | 54 | 36763 → 443 [ACK] Seq=1 Ack=1 Win=17160 Le |
| 11 | 0.044045 | 173.194.116.143 | 192.168.1.100 | TCP | 66 | 80 → 36762 [SYN, ACK] Seq=0 Ack=1 Win=4290 |

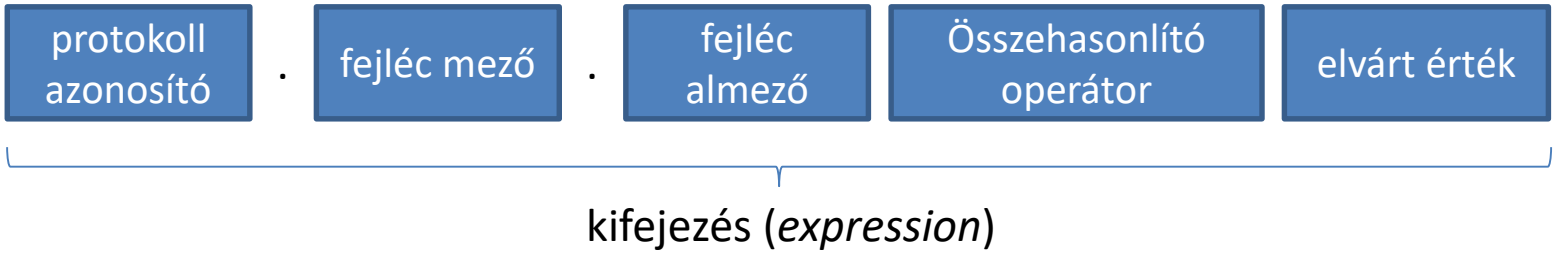
```
> Frame 2: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface \Device\NPF_{80399619-A00C-47C2-82E3-12480AEA61CB}, id 0
> Ethernet II, Src: HonHaiPr_6d:48:8b (cc:af:78:6d:48:8b), Dst: Tp-LinkT_1c:6b:9a (00:27:19:1c:6b:9a)
> Internet Protocol Version 4, Src: 192.168.1.100, Dst: 173.194.116.143
v Transmission Control Protocol, Src Port: 36761, Dst Port: 80, Seq: 0, Len: 0
    Source Port: 36761
    Destination Port: 80
    [Stream index: 1]
    [TCP Segment Len: 0]
    Sequence Number: 0    (relative sequence number)
    Sequence Number (raw): 3852632400
    [Next Sequence Number: 1    (relative sequence number)]
    Acknowledgment Number: 0
    Acknowledgment number (raw): 0
```

# Wireshark


http.accept == "image/webp,*/*;q=0.8"
No. Time Source Destination Protocol Length Info
244 6.265827 192.168.1.100 157.181.161.79 HTTP 576 GET /karunkro
245 6.267089 192.168.1.100 157.181.161.79 HTTP 556 GET /SiteColl
246 6.270616 192.168.1.100 157.181.161.79 HTTP 570 GET /Style%20
250 6.290167 192.168.1.100 157.181.161.79 HTTP 570 GET /karunkro
252 6.295020 192.168.1.100 157.181.161.79 HTTP 566 GET /Style%20
256 6.306715 192.168.1.100 157.181.161.79 HTTP 564 GET /Style%20
257 6.314244 192.168.1.100 157.181.161.79 HTTP 557 GET /SiteColl
258 6.315453 192.168.1.100 157.181.161.79 HTTP 566 GET /Style%20
260 6.323381 192.168.1.100 157.181.161.79 HTTP 558 GET /Style%20
264 6.333367 192.168.1.100 157.181.161.79 HTTP 585 GET /SiteColl
265 6.334840 192.168.1.100 157.181.161.79 HTTP 585 GET /SiteColl
GET /Style%20Library/ik/ikonok/ikon_esemeny.png HTTP/1.1\r\n
Host: www.inf.elte.hu\r\n
Connection: keep-alive\r\n
Accept: image/webp,*/*;q=0.8\r\n
User-Agent: Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/37.0.2062.12
Referer: http://www.inf.elte.hu/Lapok/kezdolap.aspx\r\n
Accept-Encoding: gzip,deflate,sdch\r\n
Accept-Language: hu-HU,hu;q=0.8,en-US;q=0.6,en;q=0.4\r\n
If-None-Match: "{7F0C9305-B4F5-4916-BA17-FF42631F52C4},6"\r\n
If-Modified-Since: Fri, 22 Feb 2008 09:53:52 GMT\r\n
\r\n
[Full request URI: http://www.inf.elte.hu/Style%20Library/ik/ikonok/ikon_esemeny.png]
[HTTP request 4/5]

# Wireshark

- Korábban rögzített adatok elemzésére szolgál.
- Szűrés felépítése:




- Operátorok: or, and, xor, not
- Példa: tcp.flags.ack==1 and tcp.dstport==80

# Szűrési feladatok 1 - HTTP

A http_out.pcapng felhasználásával állomány felhasználásával válaszolja meg az alábbi kérdéseket:

1. Milyen oldalakat kértek le a szűrés alapján? Milyen böngészőt használtak hozzá?
2. Hány darab képet érintett a böngészés? (Segítség: *webp.*)
3. Hány olyan erőforrás volt, amelyet nem kellett újra töltenie a böngészőnek? Mely oldalakat érintette ez?
4. Volt-e olyan kérés, amely titkosított kommunikációt takar? (Segítség: *SSL/TLS.*) Kövesse végig az első TCP folyamát. Mit tud kideríteni a kommunikációról?

# Szűrési feladatok 1 - HTTP

A http_out.pcapng felhasználásával állomány felhasználásával válaszolja meg az alábbi kérdéseket:

1. Milyen oldalakat kértek le a szűrés alapján? Milyen böngészőt használtak hozzá?
   - http.request.method=="GET"
2. Hány darab képet érintett a böngészés?
   - http.accept == "image/webp,*/*;q=0.8"
3. Hány olyan erőforrás volt, amelyet nem kellett újra töltenie a böngészőnek? Mely oldalakat érintette ez?
   - *http.response.code == 304* (to.ttk.elte.hu és www.inf.elte.hu)
4. Volt-e olyan kérés, amely titkosított kommunikációt takar? (Segítség: *SSL/TLS.*) Kövesse végig az első TCP folyamát. Mit tud kideríteni a kommunikációról?
   - tcp.dstport==443

# Szűrési feladatok 2 - DNS

- A dns_out.pcapng felhasználásával állomány felhasználásával válaszolja meg az alábbi kérdéseket:
  1. Hány domén név feloldást kezdeményeztek a szűrés alapján? Mely domén nevek voltak ezek?
  2. Válaszon ki 3 darab különböző domén nevet, és keresse meg a válasz csomagokat hozzájuk? Hány darab válasz van az egyes kérésekre? (Segítség: *ID.*)
  3. Hány olyan névfeloldás volt, amelyre több válasz is érkezett?
  4. Volt-e iteratív lekérdezés a szűrésben? Ha igen, akkor mennyi? Ha nem, akkor mi lehet a magyarázat?

# Szűrési feladatok 3 - NEPTUN

- A neptun_out.pcapng felhasználásával állomány felhasználásával válaszolja meg az alábbi kérdéseket:
  1. Milyen oldalakat kértek le a szűrés alapján? Milyen böngészőt használtak hozzá?
  2. Hány darab `SSL/TLS` protokollt használó csomag van? Az elsőn kövesse végig a kommunikációt. Minden működési elvnek megfelelően lezajlott?
  3. Kezdeményezett-e megszakítást a szerver a kommunikáció során?
  4. Kideríthető-e, hogy milyen kommunikáció folyt a szerver és a kliens között? Esetleg megtippelhető-e a használt böngésző típusa?

# HTTP vs HTTPS

- Nyissuk meg a sample3.pcapng fájlt!
- Keressük olyan POST kérést, amely login oldalra vezet!
- Nézzük meg a POST hívás paramétereit!

Áramkörkapcsolt hálózatok

# BEADANDÓ I. (1 PONT)

# Feladat

Adott a cs1.json, ami tartalmazza egy irányítatlan gráf leírását. A gráf végpont (end-points) és switch (switches) csomópontokat tartalmaz. Az élek (links) kapacitással rendelkeznek (valós szám). Tegyük fel, hogy egy áramkörkapcsolt hálózatban vagyunk és valamilyen RRP-szerű erőforrás foglaló protokollt használunk. Feltesszük, hogy csak a linkek megosztandó és szűk erőforrások. A json tartalmazza a kialakítható lehetséges útvonalakat (possible-cicuits), továbbá a rendszerbe beérkező, két végpontot összekötő áramkörigényeket kezdő és vég időponttal. A szimuláció a t=1 időpillanatban kezdődik és t=duration időpillanatban ér véget.

Készíts programot, ami leszimulálja az erőforrások lefoglalását és felszabadítását a JSON fájlban megadott topológia, kapacitások és igények alapján!

**Script paraméterezése**: python3 client.py cs1.json

**A program kimenete**:

<esemény sorszám. <esemény név>: <node1><-><node2> st:<szimuálciós idő> [- <sikeres/sikertelen>]

Pl.:

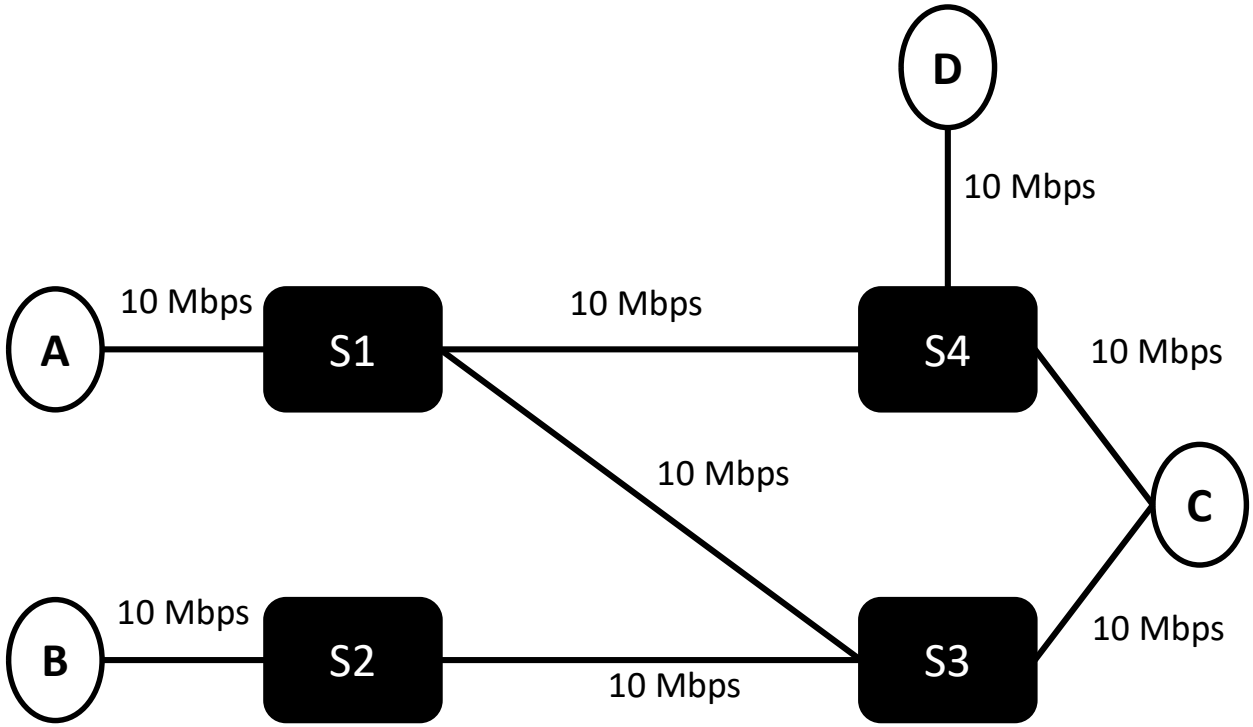
1. igény foglalás: A<->C st:1 – sikeres
2. igény foglalás: B<->C st:2 – sikeres
3. igény felszabadítás: A<->C st:5
4. igény foglalás: D<->C st:6 – sikeres
5. igény foglalás: A<->C st:7 – sikertelen

…

**Leadás:** A program leadása a TMS rendszeren keresztül .zip formátumban, amiben egy client.py szerepeljen!

**Határidő: kb 3 hét múlva TMS-ben (tms.inf.elte.hu)**

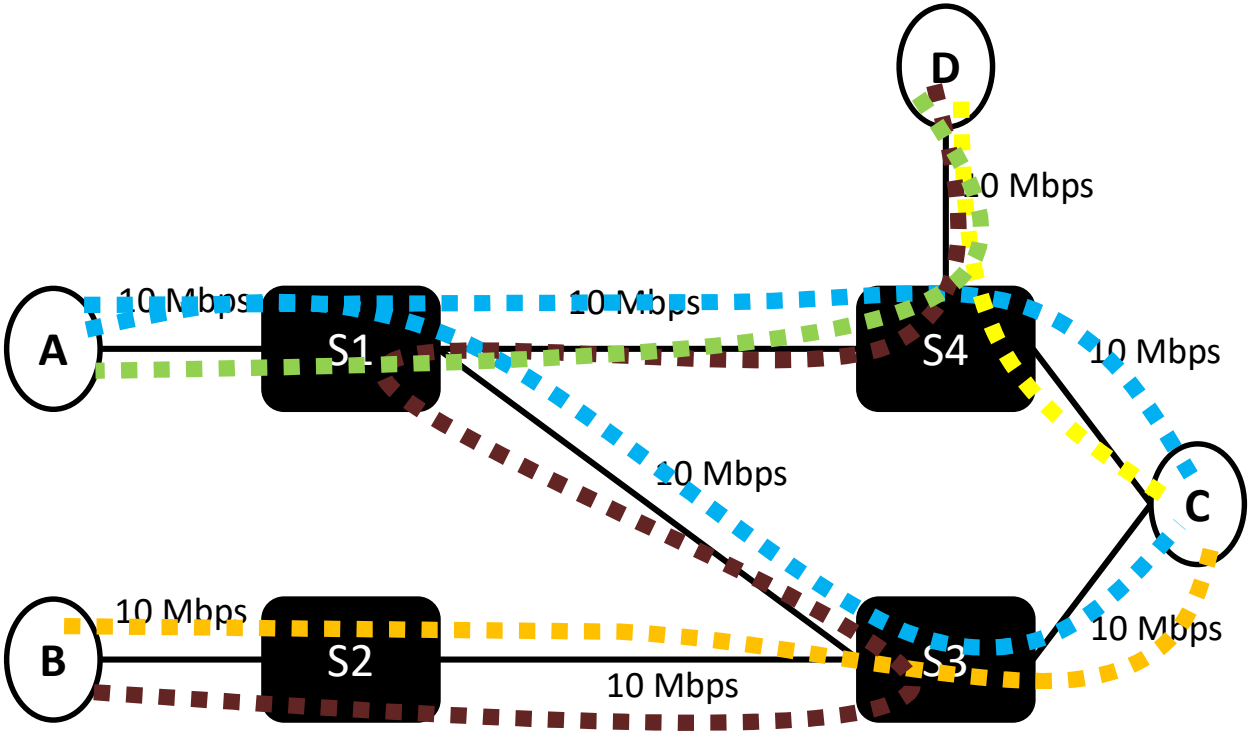
# Topológia – cs1.json




**Irányítatlan legyen a gráf!!!**

```
"end-points": [ "A", "B", "C", "D" ],
"switches": [ "S1", "S2", "S3", "S4" ],
"links" : [
        {
                "points" : [ "A", "S1" ],
                "capacity" : 10.0
        },
        {
                "points" : [ "B", "S2" ],
                "capacity" : 10.0
        },
        {
                "points" : [ "D", "S4" ],
                "capacity" : 10.0
        },
        {
                "points" : [ "S1", "S4" ],
                "capacity" : 10.0
        },
        {
                "points" : [ "S1", "S3" ],
                "capacity" : 10.0
        },
        {
                "points" : [ "S2", "S3" ],
                "capacity" : 10.0
        },
        {
                "points" : [ "S4", "C" ],
                "capacity" : 10.0
        },
```

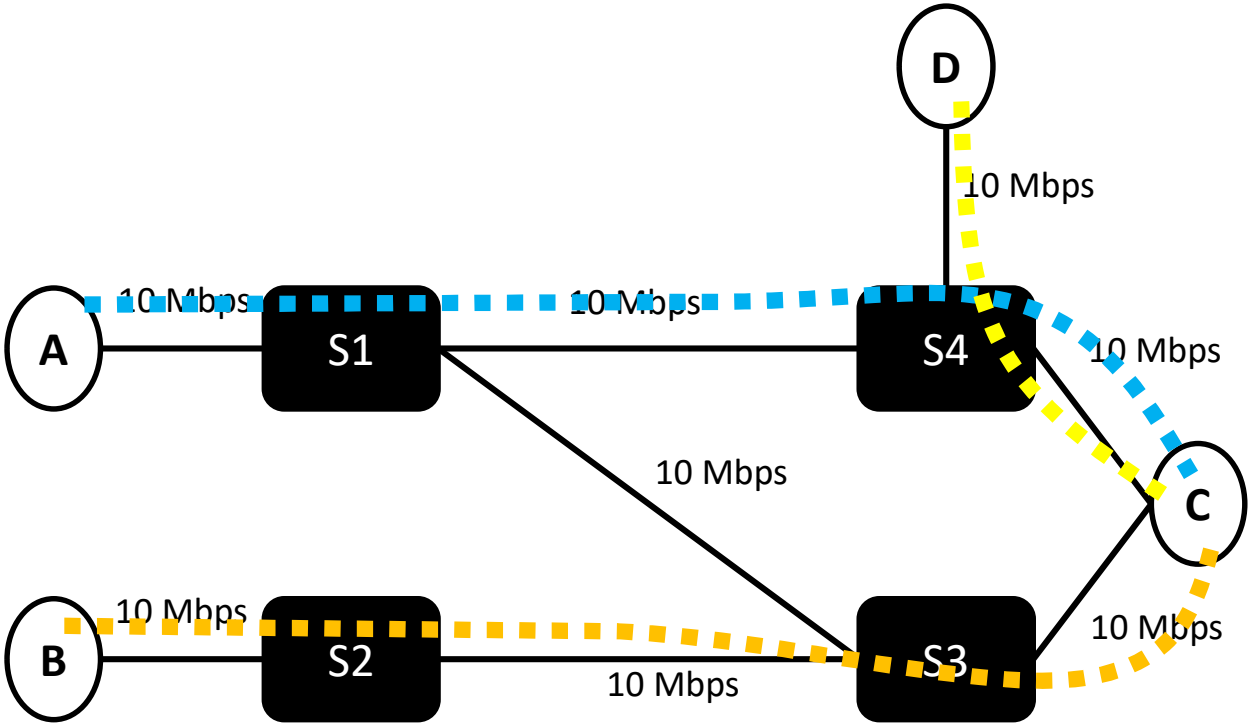
# Lehetséges áramkörök – cs1.json




```
"possible-cicuits" : [
        ["D", "S4", "C"],
        ["C", "S4", "D"],
        ["A", "S1", "S4", "C"],
        ["A", "S1", "S3", "C"],
        ["C", "S4", "S1", "A"],
        ["C", "S3", "S1", "A"],
        ["B", "S2", "S3", "C"],
        ["C", "S3", "S2", "B"],
        ["B", "S2", "S3", "S1", "A"],
        ["A", "S1", "S3", "S2", "B"],
        ["D", "S4", "S1", "S3", "S2", "B"],
        ["B", "S2", "S3", "S1", "S4", "D"],
        ["A", "S1", "S4", "D"],
        ["D", "S4", "S1", "A"]
],
```

**Irányítatlan legyen a gráf!!!**

# Igények – cs1.json




```
"simulation" : {
        "duration" : 11,
        "demands" : [
                {
                        "start-time" : 1,
                        "end-time" : 5,
                        "end-points" : ["A", "C"],
                        "demand" : 10.0
                },
                {
                        "start-time" : 2,
                        "end-time" : 10,
                        "end-points" : ["B", "C"],
                        "demand" : 10.0
                },
                {
                        "start-time" : 6,
                        "end-time" : 10,
                        "end-points" : ["D", "C"],
                        "demand" : 10.0
                }
        ]
}
```

**Irányítatlan legyen a gráf!!!**

Kiegészítő (régi) anyag:

shell parancsok

# SUBPROCESS

# Subprocess hívások és shell parancsok

**Ha nem érdekes az output:**

```
import subprocess
subprocess.call(['df', '-h']) # új verziókban run(…) //linux
subprocess.call(['dir', '/B'], shell=True) # új verziókban run(…)  //windows
```

**Ha érdekes az output:**

```
import subprocess
p = subprocess.Popen(["echo", "hello world"], stdout=subprocess.PIPE)

print(p.communicate()) # eredménye egy tuple (stdout, stderr)

# ('hello world', None)
```

**Néha a shell=True argumentum is kell, nézd meg a doksit!!! (pl. Windowson)**

**Hasznos segédletek:**

**https://docs.python.org/3/library/subprocess.html**

**https://www.pythonforbeginners.com/os/subprocess-for-system-administrators**

# subprocess – várakozás a process végére

**A process állapotának lekérdezése: poll**

```
from subprocess import PIPE, Popen
import time

p1 = Popen(["ping", '-n', '5', 'berkeley.edu'], stdout=PIPE)

while p1.poll()==None:
       print(" még fut " )
       time.sleep(1)
```

**A process végének megvárása: wait – a communicate is megvárja a végét…**

```
p1 = Popen(["ping", '-n', '20', 'berkeley.edu'], stdout=PIPE)

p1.wait() # várakozás a végére
```

subprocess

# PÁRHUZAMOSÍTÁS

# subprocess - párhuzamosítás

```
from subprocess import PIPE, Popen
from time import sleep

cmd = ["timeout","5"]       #windows
process = []
for i in range(4):     #max active process
    p = Popen(cmd, stdout=PIPE, stderr=PIPE)
    process.append(p)
ures = False
while (not ures):
    fut = 0
    i = 0
    while(i < len(process)):
        if not process[i] is None:
            if process[i].poll() == None:
                fut +=1
            else:
                print(p.communicate())
                process[i] = None
        i += 1
    if (fut > 0):
        print("Meg futnak")
        sleep(1)
    else:
        ures = True
```

# SZORGALMI FELADAT

# Ping / Traceroute - Leírás

Az Alexa-top-1M adathalmaz tartalmazza a legnépszerűbb 1 millió website domain nevét népszerűségi sorrendben:

http://s3.amazonaws.com/alexa-static/top-1m.csv.zip

Válasszuk ki az első és utolsó 10 nevet a listából, írjunk egy python programot, ami végig megy a leszűkített 20 elemű listán és minden címre lefuttatja a traceroute és ping toolokat, majd az eredményeket rendezett formában két fájlba írja! Ld. subprocess!!! Lehetőség szerint ne az egyetemi hálózaton futassuk az adatbegyűjtést!

**Script paraméterezése:** python3 client.py hosts.csv

**Traceroute paraméterek**: max. 30 hopot vizsgáljunk # tracert /h 30 <target_name> (a default beállítás is 30, de azért írjátok bele)

**Ping paraméterek**: 10 próba legyen # ping /n 10 <target_name>

Kimeneti fájlok (ld. következő dia):

traceroute.json

ping.json

A párhuzamos futtatás esetén vigyázzunk és limitáljuk a processek maximális számát!!!

# traceroute.json

```
traceroute.json:
        {
                "date" : "20180916",
                "system" : "windows",
                "traces" : [
                        {
                          "target" : "www.valami.com",
                          "output" : "Tracing route to www. . . "
                        },
                        …
                ]
        }
```

```
date: Kérjétek le az aktuális dátumot.      # time.strftime(%Y%m%d)
system: Kérjétek le az aktuális rendszert.  # platform.system()
traces: target: A CSV-ból kapott érték pl. youtube.com
        output: A Popen tracert-ból kapott tuple első része.
```

# ping.json

```
ping.json:
        {
                "date" : "20180916",
                "system" : "linux",
                "pings" : [
                        {
                          "target" : "www.valami.com",
                          "output" : „Pinging www. . . "
                        },
                        …
                ]
        }
```

# ÓRAI FELADAT

# Feladat

- Készítsünk egy olyan python kódot, ami csak annyi processzt használ, amennyi mag van a gépünkben. A program vizsgálja meg hogy Windowson vagy Linuxon fut. A subprocessek a következő parancsot futtassák: echo 'Windows' / echo 'Linux’
- os.cpu_count()
- platform.system()

**VÉGE**
**KÖSZÖNÖM A FIGYELMET!**